

Project Executable Files & Deployment Specification

Date:	3 October 2026	Team ID:	UNM1611SHARVESHGRAM-SWTID-2026-8110
Project Name:	AI BlogNest API	Maximum Marks:	3 Marks

Submission Checklist:

This document verifies that all executable and deployable deliverables for **AI BlogNest API** are provided and documented for evaluator execution and grading.

Step 1: Submission Checklist Table

S.No	Item to Submit	Submitted	File / Location Reference
1	Complete source code (all files and folders)	Yes	Code Files/src/ (controllers, models, routes, services)
2	README / Setup Guide (execution instructions)	Yes	Code Files/README.md
3	Dependency manifest (package.json / lock)	Yes	Code Files/package.json & package-lock.json
4	Database schema & ODM configuration	Yes	src/models/User.js, src/models/Blog.js, src/config/db.js
5	Environment configuration file (.env.example / .env)	Yes	Code Files/.env (PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY)
6	Deployed application / Localhost server URL	Yes	http://localhost:5000 / Cloud Production Ready
7	API Testing Suite / Postman / Thunder Client collection	Yes	Code Files/thunder-client-ai-blognest-api.postman_collection.json
8	Environment variables for testing client	Yes	Code Files/thunder-client-env.json
9	Test files and test result verification	Yes	Performance Testing.pdf & Thunder Client logs
10	Project Demonstration Video walkthrough	Yes	Ai Blog Nest.mp4 (36.4 MB high-res walkthrough video)

Project Executable Files — File & Folder Structure (Part 1)

Step 2: File / Folder Structure Directory Tree:

AI-BLOGNEST-API REPOSITORY STRUCTURE

```

■■■■ Code Files/ ■■■■ src/ ■■■■ config/ ■■■■ db.js # MongoDB Mongoose connection handler ■■■■
controllers/ ■■■■ aiController.js # Gemini blog generation, summarization & category classifier ■■■■
authController.js # User register, login & profile controller ■■■■ blogController.js # CRUD blog actions with
author ownership validation ■■■■ middleware/ ■■■■ authMiddleware.js # JWT Bearer token authentication guard
■■■■ errorHandler.js # Centralized Express error handler ■■■■ models/ ■■■■ Blog.js # Mongoose
Blog schema (title, content, category, author) ■■■■ User.js # Mongoose User schema (name, email, password,
timestamps) ■■■■ routes/ ■■■■ aiRoutes.js # Routes for /api/ai/generate-blog and /api/ai/summarize ■■■■
■■■■ authRoutes.js # Routes for /api/auth/register, /login, /profile ■■■■ blogRoutes.js # Routes for /api/blogs
CRUD endpoints ■■■■ services/ ■■■■ geminiService.js # Google Generative AI (@google/generative-ai) wrapper
■■■■ app.js # Express application setup, CORS, JSON, Morgan, Routes ■■■■ server.js # Server entry point
listening on PORT (5000) ■■■■ .env # Environment secrets (PORT, MONGO_URI, JWT_SECRET, GEMINI_KEY) ■■■■
.gitignore # Git ignore rules for node_modules and .env ■■■■ package.json # Dependencies, metadata, and start/dev
scripts ■■■■ package-lock.json # Exact dependency lockfile ■■■■ README.md # Complete setup instructions,
endpoints and API docs ■■■■ thunder-client-ai-blognest-api...json # Automated Postman/Thunder Client collection (10
requests) ■■■■ thunder-client-env.json # Thunder Client environment variables (base_url, token) ■■■■ Ai Blog
Nest.mp4 # Comprehensive video walkthrough of the API (36.4 MB)
  
```

Core Source Files Breakdown:

File Path	Lines / Size	Primary Functionality
src/server.js	14 lines	Loads dotenv, establishes MongoDB connection via connectDB(), boots HTTP server.
src/app.js	23 lines	Initializes Express, mounts CORS, JSON body parser, Morgan logging, routes, error middleware.
src/controllers/aiController.js	85 lines	Implements generateBlog (with prompt constraints), summarizeBlog, cleanGeneratedText, inferCategory.
src/controllers/blogController.js	82 lines	Implements createBlog, getAllBlogs (with populate), getBlogById, updateBlog, deleteBlog.
src/controllers/authController.js	75 lines	Implements register (bcrypt hash), login (password compare & JWT generation), and profile.

Project Executable Files — Deployment & Environment Details

Step 3: Deployment & Access Details:

Hosted / Deployed URL:	http://localhost:5000 (Local Dev) / https://ai-blognest-api.onrender.com (Production)
Demo Login Credentials:	Email: john@example.com Password: 123456 (Pre-seeded demo user)
Hosting Platform:	Node.js Runtime Environment / Render Cloud / MongoDB Atlas Cloud Cluster
Repository Link:	https://github.com/sharvesh007ram/ai-blognest-api (Local workspace deliverables)
Demo Video Walkthrough:	Ai Blog Nest.mp4 (Included in root directory, 36,491,678 bytes, full feature demo)

Environment Configuration Specifications (.env):

Variable Key	Example Value / Format	Security Role & Purpose
PORT	5000	Defines the local TCP network port where Express listens for incoming HTTP requests.
MONGO_URI	mongodb+srv://.../Blog_nest	Encrypted TLS connection string to MongoDB Atlas database cluster.
JWT_SECRET	YOUR_JWT_SECRET_KEY	Private cryptographic secret key used to sign and verify HMAC-SHA256 JWT tokens.
GEMINI_API_KEY	AQ.Ab8RN6KM...xdlQ	Google Cloud AI Studio secret key authorizing generative LLM inference requests.
GEMINI_MODEL	gemini-1.5-flash / gemini-2.5-flash	Specifies the generative model engine; defaults to gemini-1.5-flash if omitted.

Project Executable Files — Run Instructions & Known Issues

Step 4: Step-by-Step Local Run Instructions:

Step 1 — Prerequisites Installation:

Ensure Node.js (version 18 or higher) and npm are installed on the evaluator system. Verify with `node -v` and `npm -v`.

Step 2 — Navigate to Project Directory:

Open terminal in the codebase folder:

```
cd "Code Files-20260928T090059Z-1-001/Code Files"
```

Step 3 — Install Dependencies:

Install all required Node.js libraries by executing:

```
npm install
```

Step 4 — Verify Environment Variables:

Ensure `.env` file contains valid parameters: `PORT=5000`, `MONGO_URI`, `JWT_SECRET`, and `GEMINI_API_KEY`.

Step 5 — Start the Server:

Run the development server with live reload or production start:

```
npm run dev (or npm start)
```

The terminal should output: Server running on port 5000 and MongoDB connected: cluster0...

Step 6 — Execute Endpoint Verification:

Open VS Code Thunder Client or Postman. Import `thunder-client-ai-blognest-api.postman_collection.json`. Run the collection to verify all 10 endpoints passing with 200/201 status codes!

Step 5: Known Issues / Limitations & Workarounds:

S.No	Known Limitation / Issue	Workaround / Mitigation Strategy	Status
1	Gemini Free Tier Rate Limits (15 RPM)	Implemented exponential backoff retry and fallback to gemini-1.5-flash for maximum token efficiency.	Mitigated
2	Cold Start Latency on Free MongoDB Tier	Mongoose connection established at server boot (server.js) with keep-alive socket options.	Resolved
3	JWT Token Expiration (7-Day TTL)	Client must re-login to obtain fresh token; clear 401 Unauthorized message guides the user.	Expected